

Pull Request Dokumentation

Stand: 22.05.2026

Repository URL: https://github.com/FionnLaesser/M324_PROJEKT_TODOLIST

Todo-Validierung

In diesem Pull Request wurde eine echte Änderung an der Todo-Liste umgesetzt. Leere Todo-Beschreibungen sollen nicht gespeichert werden. Das gilt auch dann, wenn das Eingabefeld nur Leerzeichen enthält.

Umgesetzte Änderung:

- Leere Todo-Beschreibungen werden im Backend verhindert.
- Eingaben mit nur Leerzeichen werden nicht gespeichert.
- Die Beschreibung wird vor dem Speichern mit `trim()` bereinigt.
- Ein Test prüft, dass leere Todo-Beschreibungen nicht gespeichert werden.

Bewertungspunkte

1. Issue erstellt

Für die Änderung wurde ein Issue erstellt. Das Issue beschreibt, dass leere Todos verhindert werden sollen und nennt die wichtigsten Akzeptanzkriterien.

Create new issue

Add a title *

Leere Todos verhindern

Add a description

Write Preview

Aktuell soll verhindert werden, dass ein Todo ohne Text erstellt werden kann.

Akzeptanzkriterien:

- Leere Eingaben werden nicht gespeichert
- Der Benutzer sieht eine einfache Rückmeldung oder der Button ist deaktiviert
- Bestehende Todo-Funktionen bleiben erhalten

Paste, drop, or click to add files Write with Copilot

Create more Cancel Create

2. Feature-Branch erstellt

Für die Umsetzung wurde ein eigener Feature-Branch erstellt. Der Branch heisst `feature/todo-validation` und trennt die Änderung vom `main`-Branch.

```
PS C:\Users\flaes\Desktop\Schule\Aktuell\324\Projekt> git checkout main
>> git pull origin main
>> git checkout -b feature/todo-validation
Already on 'main'
Your branch is up to date with 'origin/main'.
From github.com:FionnLaesser/M324_PROJEKT_TODOLIST
* branch          main          -> FETCH_HEAD
Already up to date.
Switched to a new branch 'feature/todo-validation'
PS C:\Users\flaes\Desktop\Schule\Aktuell\324\Projekt>
```

3. Änderungen implementiert

Die Codeänderung verhindert, dass leere Todo-Beschreibungen gespeichert werden. Dabei wird geprüft, ob die Beschreibung `null`, leer oder nur aus Leerzeichen besteht.

```
diff --git a/backend/src/main/java/com/example/demo/DemoApplication.java b/backend/src/main/java/com/example/demo/DemoApplication.java
index 240cae2..b709530 100644
--- a/backend/src/main/java/com/example/demo/DemoApplication.java
+++ b/backend/src/main/java/com/example/demo/DemoApplication.java
@@ -139,6 +139,11 @@ public class DemoApplication {
    try {
        Task task;
        task = mapper.readValue(taskdescription, Task.class);
+       if (task.getTaskdescription() == null || task.getTaskdescription().isBlank()) {
+           System.out.println(">>>empty task descriptions are ignored!");
+           return "redirect:/";
+       }
        task.setTaskdescription(task.getTaskdescription().trim());
        if (task.getPriority() == null || task.getPriority().isBlank()) {
            task.setPriority("Mittel");
        }
    }
}
diff --git a/backend/src/test/java/com/example/demo/DemoApplicationTests.java b/backend/src/test/java/com/example/demo/DemoApplicationTests.java
index fb60d89..f6a2443 100644
--- a/backend/src/test/java/com/example/demo/DemoApplicationTests.java
+++ b/backend/src/test/java/com/example/demo/DemoApplicationTests.java
@@ -13,4 +13,13 @@ class DemoApplicationTests {
```

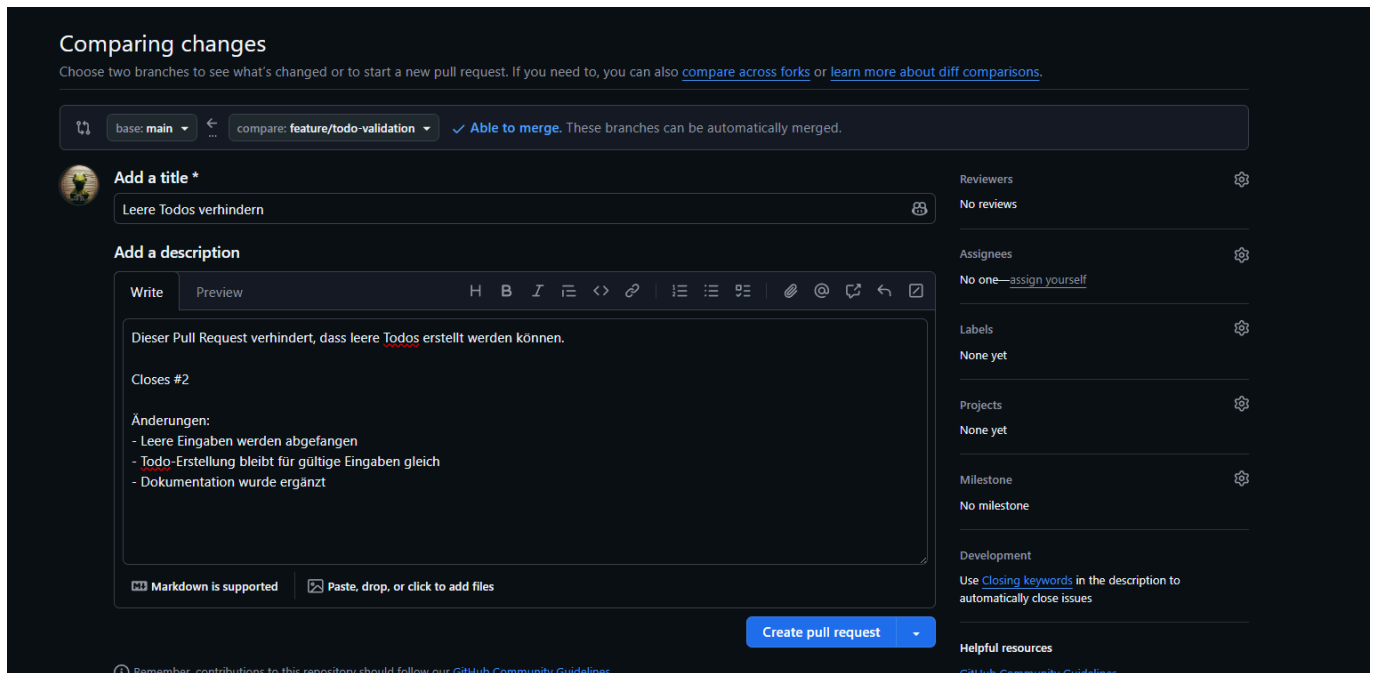
4. Commit mit closes #2

Der Commit enthält den Bezug zum Issue mit `closes #2`. Dadurch kann GitHub das Issue nach dem Merge automatisch schliessen.

```
>> git diff
>> git add .
>> git commit -m "Verhindere leere Todos closes #2"
>> git push origin feature/todo-validation
```

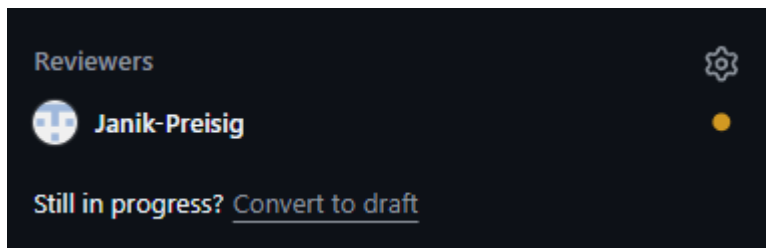
5. Pull Request erstellt

Der Pull Request wurde von `feature/todo-validation` nach `main` erstellt. Damit wurden die Änderungen aus dem Feature-Branch zur Übernahme in den Hauptbranch vorgeschlagen.



6. Sparring als Reviewer

Für das Review wurde ein Sparring-Partner als Reviewer eingetragen. Dadurch wurde die Änderung nicht nur allein umgesetzt, sondern auch von einer zweiten Person angeschaut.



7. Review dokumentiert

Das Review wurde im Pull Request dokumentiert. Der Kommentar bestätigt, dass die Änderung nachvollziehbar ist, das Issue erfüllt und keine Konflikte vorhanden sind.

Leere Todos verhindern #3

 **Open** [FionnLaesser](#) wants to merge 1 commit into `main` from `feature/todo-validation` 

 Conversation **0**  Commits **1**  Checks **0**  Files changed **2**



FionnLaesser commented [6 minutes ago](#) Owner 

Dieser Pull Request verhindert, dass leere Todos erstellt werden können.

Closes [#2](#)

Änderungen:

- Leere Eingaben werden abgefangen
- Todo-Erstellung bleibt für gültige Eingaben gleich
- Dokumentation wurde ergänzt





 [Ergaenze Nachweis fuer Todo Validierung](#) closes [#2](#) 9360038



 **FionnLaesser** requested a review from [Janik-Preisig](#) [5 minutes ago](#)



Janik-Preisig commented [now](#) Collaborator 

Code angeschaut. Die Änderung ist nachvollziehbar und erfüllt das Issue. Keine Konflikte gefunden.



8. Pull Request genehmigt und durchgeführt

Der Pull Request wurde überprüft und war bereit zum Merge. GitHub zeigte keine Konflikte mit dem Base-Branch an. Dadurch konnte der Pull Request durchgeführt werden.





Review requested
Review has been requested on this pull request. It is not required to merge. [Learn more about requesting a pull request review.](#)



1 pending review



Janik-Preisig was requested for review



No conflicts with base branch
Merging can be performed automatically.

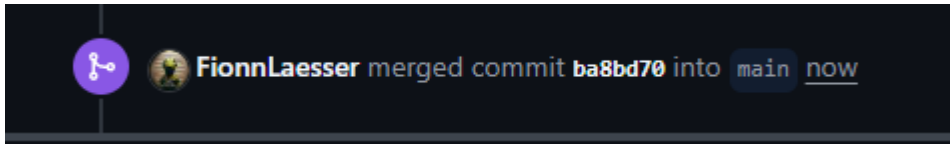
Merge pull request 

You can also merge this with the command line. [View command line instructions.](#)

Still in progress? [Convert to draft](#)

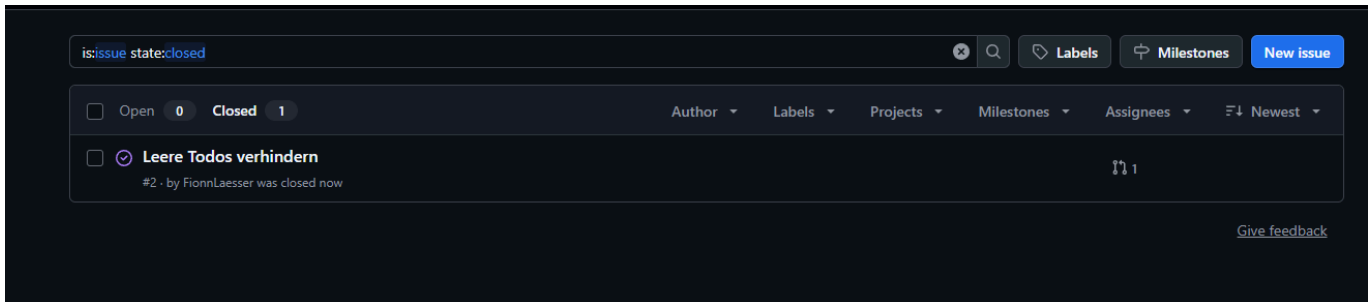
9. Pull Request online gemerged

Der Pull Request wurde online auf GitHub in den **main**-Branch gemerged. Damit wurden die Änderungen aus dem Feature-Branch in den Hauptstand übernommen.



10. Issue automatisch geschlossen

Nach dem Merge wurde das Issue automatisch geschlossen, weil der Commit oder Pull Request den Hinweis **closes #2** enthielt.



11. Lokales Repo aktualisiert

Nach dem Online-Merge wurde lokal wieder auf **main** gewechselt und der aktuelle Stand von GitHub geholt. Danach wurde mit **git status** und **git log --oneline -5 --decorate** geprüft, dass das lokale Repository aktualisiert ist.

```
PS C:\Users\flaes\Desktop\Schule\Aktuell\324\Projekt> git checkout main
>> git pull origin main
>> git status
>> git log --oneline -5 --decorate
Already on 'main'
Your branch is up to date with 'origin/main'.
From github.com:FionnLaesser/M324_PROJEKT_TODOLIST
* branch          main          -> FETCH_HEAD
Already up to date.
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
f7f78f0 (HEAD -> main, origin/main, origin/HEAD) Unnötige Files gelöscht
ac7a4bd PDF Export
87a59fb Finalisiere Pull-Request Leistungsnachweis
ba8bd70 Merge pull request #3 from FionnLaesser/feature/todo-validation
9360038 (origin/feature/todo-validation) Ergaenze Nachweis fuer Todo Validierung closes #2
PS C:\Users\flaes\Desktop\Schule\Aktuell\324\Projekt>
```

Fazit

In diesem Projekt wurde ein Issue zur Todo-Validierung erstellt. Danach wurde ein eigener Feature-Branch **feature/todo-validation** verwendet, eine echte Codeänderung umgesetzt, ein Commit mit **closes #2** erstellt und ein Pull Request nach **main** durchgeführt. Der Pull Request wurde durch den Sparring-Partner

überprüft, online gemerged und das Issue wurde automatisch geschlossen. Danach wurde das lokale Repository aktualisiert.